

國立臺灣大學

資訊管理導論期末專案書面報告

堯順工程行智慧儀表板 與 LINE Bot 整合系統

第三組

組員 劉起佑、陳鼎元、蔡承叡、鍾心哲、蔡冠毅

合作對象 堯順工程行

專案類型 小型工程行訂單、出貨、收款與營運分析系統

Dashboard Repository <https://github.com/cpcap1214/IM2008>

LINE Bot Repository <https://github.com/OogaryoO/yaoshun>

撰寫日期 2026 年 6 月

目錄

1	報告規範與評分項目對照	3
2	專案背景與公司介紹	3
2.1	合作公司背景	3
2.2	專案動機	4
3	訪談內容與痛點分析	4
3.1	訪談摘要	4
3.2	現行流程	4
3.3	核心痛點	5
3.3.1	痛點一：ERP 流程過重，與小型工程行日常不匹配	5
3.3.2	痛點二：訂單與配送狀態分散	5
3.3.3	痛點三：帳款追蹤混亂	5
3.3.4	痛點四：營運資料無法形成決策依據	5
3.4	專案範圍取捨	5
4	商業模式畫布	6
5	痛點解決方案與系統目標	7
5.1	整體解方	7
5.2	開發目標	8
6	系統設計與架構	9
6.1	整體架構	9
6.2	技術選型	10
6.3	資料庫設計	10
6.4	訂單狀態與正確性規則	12
6.5	跨 Repository 協作	13
7	系統功能與使用流程	13
7.1	Dashboard 功能	13
7.2	LINE Bot 使用流程	14
7.3	報表匯出	14

7.4	AI 智慧顧問	14
8	系統正確性與測試驗證	15
8.1	型別與資料驗證	15
8.2	測試涵蓋	15
8.3	已知限制	15
9	與既有解決方案之差異與創新性	16
9.1	與 ERP 系統比較	16
9.2	與 Excel、紙本或純 LINE 紀錄比較	16
9.3	創新性	17
10	對企業預期產生之效益	17
10.1	作業效率提升	17
10.2	帳款風險降低	17
10.3	資訊透明度提升	18
10.4	決策品質提升	18
11	專案啟發與未來展望	18
11.1	專案啟發	18
11.2	未來展望	18
12	交付內容與執行方式	19
12.1	專案交付內容	19
12.2	Dashboard 執行方式	19
12.3	LINE Bot 執行方式	19
13	結論	20

1 報告規範與評分項目對照

依據「資訊管理導論期末專案評分說明」，書面報告占期末專案總成績 50%，評分重點包含痛點分析、商業模式畫布、系統正確性、系統實用性、系統創新性與內容完整性。評分說明中特別提醒，書面報告不能只有投影片剪貼，必須比口頭簡報提供更完整的訪談脈絡、系統設計理由與效益說明。因此本報告以正式簡報內容為基礎，並補充系統架構、資料契約、使用流程與測試驗證。

表 1: 書面報告評分項目與本報告對應章節

評分項目	本報告對應內容
痛點分析	第 2 章整理訪談內容、現行流程、四項核心痛點與本專案範圍取捨。
商業模式畫布	第 3 章以商業模式畫布的觀點說明本系統如何創造、傳遞與獲取價值。
系統正確性	第 5 章說明架構、資料庫 schema、狀態轉換規則與跨 repo 契約；第 7 章補充測試與限制。
系統實用性	第 4 章痛點對解方，第 6 章詳細說明使用流程與功能如何貼近日常工作。
系統創新性	第 8 章比較 ERP、Excel/紙本與 LINE 人工紀錄，說明本系統在 LINE、儀表板與 AI 整合上的差異。
內容完整性	第 9 至第 11 章補足企業效益、未來展望、交付內容與執行方式。

2 專案背景與公司介紹

2.1 合作公司背景

本專案合作對象為堯順工程行，地點位於彰化縣埔鹽鄉。堯順工程行主要從事水泥與建材相關工程產品的製作與配送，產品包含化糞池、電線桿、陰井、涵管等。其營運型態具有明顯的小型工程行特徵：訂單多來自熟客或長期往來對象，溝通方式以 LINE 為主，現場工作涵蓋接單、生產、配送與收款，且許多資訊仰賴老闆與員工之間的口頭記憶或即時訊息。

與一般大型製造業相比，堯順工程行不需要完整而複雜的企業資源規劃系統。公司最需要的是一個能支援日常工作、降低記憶負擔、快速查詢訂單與帳款狀態的輕量工具。本專案因此將重點放在「訂單、出貨、收款、客戶歷史與營運洞察」五個面向，

設計出一套貼合小型工程行流程的智慧儀表板，並與 LINE Bot 共用 Firestore 資料庫，讓客戶與老闆能在熟悉的通訊工具中完成下單、查詢與付款回報。

2.2 專案動機

在訪談中，業主提到過去曾購買 ERP 系統，希望改善成本與收入記錄。但實際使用後發現，ERP 雖然功能完整，卻要求建立大量客戶、商品、庫存或會計資料，操作流程遠超過目前業務需求。對小型工程行而言，太完整的系統反而造成學習與維護負擔，使業主回到 LINE 訊息、人工記憶與零散表格的工作方式。

因此本專案的核心動機不是建置一套功能龐大的 ERP，而是用資訊系統解決最迫切、最常發生、最容易造成金流與溝通風險的問題：誰下了什麼訂單、是否送達、是否付款、付款方式為何、哪些客戶累積未收金額較高，以及老闆每天應優先處理哪些事項。

3 訪談內容與痛點分析

3.1 訪談摘要

本組依正式簡報所整理之訪談內容，將業主日常工作與需求歸納如下。

1. 工作通常跨日進行。業主表示，一份訂單通常不會一天完成，因此每天開始工作時需要先檢查昨天的進度，再繼續製作或安排配送。
2. 出貨責任可能分散。訂單完成後，可能由老闆本人或其他人開卡車配送給客戶，因此「是否送達」與「由誰送達」若沒有統一紀錄，很容易只停留在口頭確認。
3. 既有 ERP 過於複雜。業主原本為了記錄成本與收入購買 ERP，但系統流程繁重，甚至為了記錄訂單還需要完整客戶建檔；對目前需求而言，只用到其中少部分功能。
4. 帳款追蹤混亂。客戶付款方式包含現金、匯款、支票與賒帳，且收款的人不一定是老闆本人，導致實際收入與未收款項難以追蹤。

3.2 現行流程

堯順工程行的主要營運流程可整理為「接單、製作、配送、收款、結算」五個階段。問題不只發生在某一個單點，而是在多個階段之間的資訊交接與狀態更新。

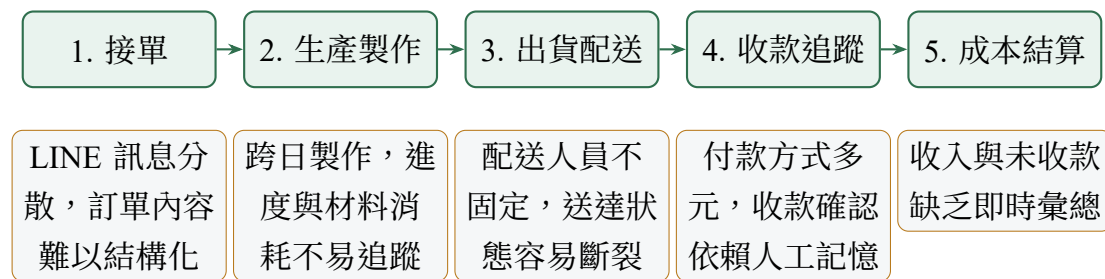


圖 1: 堯順工程行現行流程與資訊斷點

3.3 核心痛點

3.3.1 痛點一：ERP 流程過重，與小型工程行日常不匹配

業主需要的是能快速記錄訂單、配送與收款狀態的工具，而不是完整導入會計、庫存、人資、採購與生產模組。既有 ERP 對大型企業有價值，但對目前規模而言，前置建檔、欄位填寫與系統學習成本過高，導致業主不願持續使用。

3.3.2 痛點二：訂單與配送狀態分散

水泥製品與工程材料訂單常跨日處理，並可能由不同人配送。若訂單進度、配送地址、送貨人員、送達時間只存在於 LINE 對話或口頭回報，老闆難以快速回答「今天有哪些尚未送達」、「哪一筆已完成但未收款」、「某位客戶最近訂單狀況如何」等問題。

3.3.3 痛點三：帳款追蹤混亂

堯順工程行的付款方式包含現金、匯款與支票，且可能發生賒帳或由非老闆本人收款的情境。若沒有統一狀態，已付款、未付款、客戶已回報但老闆尚未確認入帳等情況會混在一起，造成現金流風險，也增加每天追帳的時間成本。

3.3.4 痛點四：營運資料無法形成決策依據

即使訂單與收款資料已經存在於訊息或零散紀錄中，業主仍缺乏整體視角，例如本月營收、未收款總額、待出貨筆數、付款方式比例、累積未收排行、久未收款客戶等。缺乏彙總與分析會使管理行動依賴直覺，而不是依據資料排序優先順序。

3.4 專案範圍取捨

正式簡報中曾提及成本不透明與材料耗損難追等議題。這些問題確實重要，但若在單一學期內同時處理 BOM、材料耗損、成本會計、訂單管理、出貨、收款與 LINE 整合，會使系統過於分散。基於訪談中業主最迫切的需求，本專案 MVP 將範圍聚焦於

訂單、出貨與帳款追蹤，並以 AI 與報表作為輔助分析。成本與材料耗損則列為未來擴充方向。

4 商業模式畫布

本章以 Business Model Canvas 描述本專案的輕量化 ERP 與 LINE Bot 整合服務如何創造、傳遞與獲取價值。畫布的重點不是取代大型 ERP，而是針對堯順工程行及相似小型工程行的日常流程，提供能快速導入、低學習成本且可持續擴充的管理工具。

表 2: 堯順工程行智慧管理系統商業模式整理

區塊	說明
關鍵夥伴	本專案以 LINE 官方帳號與 Bot 平台作為主要互動介面，並以 Firebase 與 Google Cloud 提供雲端資料儲存與系統運作環境；若需進一步提供營運分析，則可結合 Gemini AI 或本地規則引擎作為輔助工具。此外，堯順工程行老闆與現場人員提供真實流程、使用情境與需求回饋，而系統開發、導入與維護團隊則負責將需求轉化為可落地的資訊系統。
關鍵活動	本系統的核心活動包括維護訂單、商品與客戶資料，整合 LINE 訊息、下單與付款回報流程，並持續追蹤出貨、收款與待確認狀態。系統同時會產生匯出報表與 AI 營運建議，協助業主掌握整體營運情況，並依據實際使用回饋調整欄位與流程設計。
關鍵資源	支撐本系統運作的主要資源包括 Firestore 即時資料庫、Web 儀表板、LINE Bot，以及累積下來的訂單、出貨、收款與客戶歷史資料。更重要的是，系統建立在對小型工程行日常作業流程的理解之上，因此能比一般大型 ERP 更貼近現場需求。
價值主張	本專案的價值主張在於，為工程行提供一套客製化且輕量的 ERP 工具，使業主不必導入過於龐大複雜的系統，也能快速完成訂單紀錄、出貨追蹤與帳款管理。透過 LINE 自動提醒與回報機制，系統可降低人工追帳成本，並讓業主即時掌握待出貨、待收款與待確認事項；同時支援 Excel、CSV 與 PDF 匯出，以符合既有使用習慣。

區塊	說明
顧客關係	在顧客關係設計上，本系統強調低學習成本與即時互動。客戶可透過 LINE 接收付款提醒並進行回報，老闆則可在確認後更新帳務狀態，減少雙方資訊落差。整體操作方式盡量貼近日常習慣，使使用者不需要大幅改變既有流程即可逐步導入。
通路	本系統的主要通路包含接案網站、LINE Bot 與口碑推薦。其中網站負責展示系統功能與導入說明，LINE Bot 作為日常互動與通知的主要入口，而口碑推薦則有助於在同類型工程行之間擴散，形成後續導入機會。
顧客區隔	本專案主要面向小型工程行，以及那些認為大型 ERP 太重、太複雜、導入成本過高的店家。這些業者通常具有接單分散、出貨跨日、付款方式多元、帳款追蹤不易等特徵，因此更需要一套能快速落地的管理工具。
成本結構	本系統的主要成本包括 Firebase 與雲端資料庫部署費用、LINE Bot 訊息推播與串接成本，以及 AI API、規則引擎與資料分析維護等技術支出。若擴大導入規模，後續也需考慮教育訓練、資料整理與維護服務的人力成本。
收入來源	收入來源可由專案導入與客製化建置費、月租型 SaaS 使用費或維護合約構成。若未來成功擴展到多家工程行，則有機會進一步發展為可持續訂閱的服務模式，並延伸出資料分析或進階功能加值服務。

整體而言，這套商業模式的核心，不是以功能完整性與大型企業流程控制為目標，而是以「夠用、好懂、能立即改善日常管理」為優先。它把小型工程行原本依賴 LINE、口頭與人工記憶的資訊流，轉換成可追蹤、可查詢、可匯出且可分析的營運資料，並以較低的導入門檻換取實際可見的管理效益。

5 痛點解決方案與系統目標

5.1 整體解方

本專案提出三個互相連動的解方：智慧儀表板、LINE Bot 追帳與互動流程、AI 智慧顧問。

1. **智慧儀表板**：整合訂單、客戶、商品、付款與配送資料，讓老闆能在單一畫面掌握營收、未收款、待出貨與待確認事項。
2. **LINE Bot**：讓客戶可透過 LINE 查看商品、下單、查詢未付款與回報付款；老闆可查未付款清單、確認付款、幫客戶下單與標記送達。
3. **AI 智慧顧問**：串接 Gemini API，並提供規則引擎 fallback，根據訂單、客戶與商品資料產生營運提醒與分析建議。

表 3: 痛點與解決方式對應

痛點	系統解方	預期改善
ERP 過於複雜	以工程行日常流程為核心，只保留訂單、出貨、收款、客戶與洞察五個主要視圖。	降低建置與學習門檻，使業主不需理解完整 ERP 流程也能開始使用。
訂單進度分散	Firestore 統一保存訂單品項、配送地址、送達時間與送貨標籤；儀表板即時訂閱資料。	減少 LINE 訊息翻找與人工確認，讓待出貨、已送達、待收款狀態清楚可見。
帳款追蹤混亂	設計 unpaid、pending_confirmation、paid 三種付款狀態，並記錄付款方式與入帳時間。	區分「未付款」與「客戶已回報但老闆未確認」，降低誤判與漏收風險。
決策依據不足	Dashboard KPI、付款方式分析、客戶歷史、累積未收排行與 AI 建議。	讓老闆能先處理金額大、時間久或風險高的訂單，提高管理優先順序清晰度。
報表整理耗時	以 CSV、Excel、列印 PDF 匯出指定欄位。	符合業主習慣使用 xlsx 的需求，也方便對帳與留存。

5.2 開發目標

本系統設計時遵循以下目標。

- **直覺**：介面用語採用業主能理解的「待出貨、待收款、待老闆確認、標記送達、確認入帳」等業務語言。
- **輕量**：不要求一開始完整導入 ERP，只需建立商品、客戶與訂單，即可開始使用。

- **即時**：Dashboard 透過 Firestore 即時訂閱，LINE Bot 與前端寫入後能在另一端同步呈現。
- **可追蹤**：付款狀態、配送狀態、入帳時間與客戶歷史皆有結構化欄位。
- **可擴充**：Firestore schema、索引與 AI provider 皆保留未來擴充點，例如 LIFP、成本模組或其他 AI 模型。

6 系統設計與架構

6.1 整體架構

本專案由兩個 repository 組成。Dashboard 端使用 React、TypeScript、Vite、Tailwind CSS 與 shadcn/ui 建置；LINE Bot 端使用 Flask、LINE Messaging API 與 Firebase Admin SDK。兩端共用同一個 Firestore 專案，並遵守相同的資料契約。

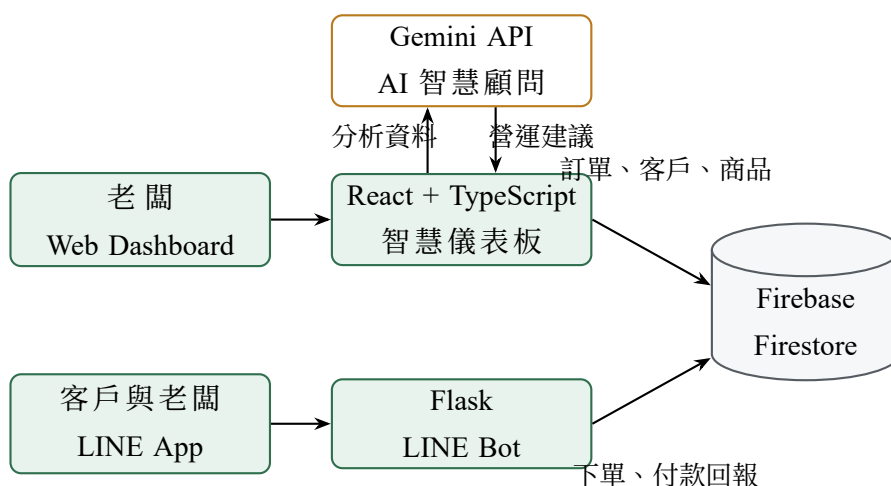


圖 2: 系統整體架構

6.2 技術選型

表 4: 主要技術與用途

技術	用途與選用理由
React 19 + TypeScript	建置前端儀表板，TypeScript 可降低資料欄位與狀態轉換錯誤。
Vite	提供快速開發伺服器與前端打包流程。
Tailwind CSS + shadcn/ui	以現代化元件建立一致且易掃描的管理介面。
Firebase Firestore	以雲端文件資料庫儲存 Users、Products、Orders，支援即時訂閱。
LINE Messaging API	讓客戶與老闆在熟悉的 LINE 介面下單、查詢與回報付款。
Flask + firebase-admin	實作 LINE Webhook 與後端 Firestore 寫入驗證。
Gemini API + 規則引擎	提供 AI 營運分析；未設定 API key 或呼叫失敗時回到本地規則。
SheetJS xlsx	產出 Excel 報表，符合業主慣用 xlsx 的需求。

6.3 資料庫設計

Firestore 以三個核心集合支撐系統運作：Users、Products、Orders。其中 Orders 是系統核心，Dashboard、LINE Bot、報表與 AI 分析都依賴此集合。

表 5: Firestore 核心資料表設計

Collection	重要欄位	說明
Users	role, displayName, phone, createdAt, notes	使用者基本資料。Dashboard 端目前只使用 boss 與 customer；舊的 driver 角色會被讀取端轉為 customer，以維持相容。

Collection	重要欄位	說明
Products	productName, spec, price, isActive	商品型錄與價格來源。Dashboard 填單與 LINE Bot 商品列表皆依此產生選項；isActive=false 的商品不顯示於客戶下單清單。
Orders	customerId, customerName, driverId, deliveryAddress, items, totalAmount, paymentStatus, paymentMethod, orderDate, deliveryDate, paidAt	訂單、配送與收款核心資料。driverId 保留欄位名稱以對齊既有 bot，但在本系統中是自由文字配送標籤，不是 Users foreign key。

圖 3 以較正式的 ER 圖方式呈現 Firestore 的概念資料模型，並標示主鍵（PK）、邏輯外鍵（FK）、衍生欄位（DER）與嵌入欄位（EMB）。需注意的是，本系統採文件資料庫設計，OrderItems 並不是獨立 collection，而是嵌在 Orders.items[] 內的陣列；商品資訊在建立訂單時會反正規化寫入訂單品項，以保留當下品名、規格與單價。

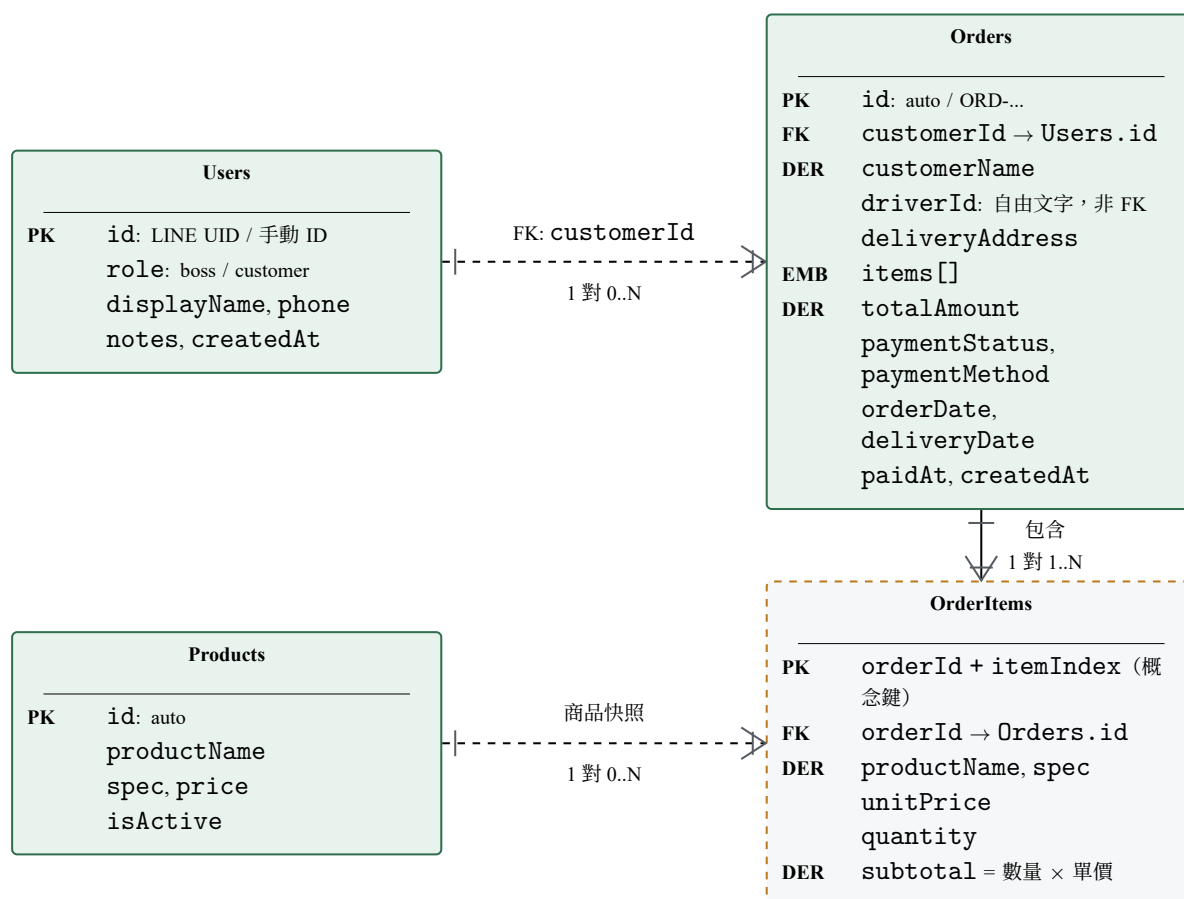


圖 3: Firestore 概念 ER 圖

6.4 訂單狀態與正確性規則

為了避免資料不一致，本系統在前端 TypeScript converter、Firestore 寫入函式與 bot 端 schema validator 中都定義狀態規則。

1. **付款狀態**：paymentStatus 只允許 unpaid、pending_confirmation、paid。
2. **付款方式**：只有狀態為 paid 時，paymentMethod 才能是 cash、transfer 或 check；其他狀態必須為 null。
3. **入帳時間**：只有轉為 paid 時寫入 paidAt，並使用 server timestamp，避免使用者本機時間不準。
4. **配送狀態**：deliveryDate 只由「標記送達」流程寫入；未送達時為 null。
5. **訂單金額**：totalAmount 必須等於所有 items[].subtotal 總和，而每筆 subtotal 應等於單價乘以數量。
6. **未指派配送**：driverId=null 代表未指派；系統不寫入「尚未指派」字串，以免查詢與顯示邏輯混淆。

6.5 跨 Repository 協作

Dashboard repository 與 LINE Bot repository 透過 Firestore 共享資料。Dashboard 端的 `src/lib/firestore.ts` 是主要型別契約來源；Bot 端的 `services/schemas.py` 則以 `data-class` 與 `validator` 重現相同 `shape`，在寫入 Firestore 前檢查欄位是否符合規則。此設計讓兩端可以獨立開發，但仍維持資料一致。

LINE Bot repository 目前主分支版本為 `7b74118`。其主要功能包含：

- 客戶：查看商品、我要下單、我的未付款、回報付款、聯絡老闆。
- 老闆：幫客戶下單、查詢客戶、客戶訂單、未付款清單、確認付款、送達。
- Rich Menu：依照老闆或客戶角色綁定不同 LINE 圖文選單。
- Firestore 寫入：建立訂單、查詢未付款、客戶回報付款、老闆確認付款、標記送達。

7 系統功能與使用流程

7.1 Dashboard 功能

Dashboard 採左側 sidebar 分頁設計，共分為總覽、填寫資料、訂單作業、客戶、洞察五個主要視圖。

表 6: Dashboard 主要視圖與功能

視圖	功能說明
總覽	呈現 KPI、今日訂單、本月已收、目前未收、待出貨、待確認與近期訂單，讓老闆開啟系統後先掌握優先事項。
填寫資料	建立或編輯訂單。支援多品項、商品下拉自動帶入規格與價格、配送地址、付款狀態與付款方式。送出後停留在表單頁，方便連續建單。
訂單作業	合併帳款與出貨管理。可依全部、待出貨、待收款、待確認、久未收款篩選；可標記送達、確認入帳、編輯、刪除與匯出報表。
客戶	顯示客戶名冊與統計，包括累積營收、目前未收、最近訂單。點選客戶可開啟歷史訂單彈窗並直接操作訂單。

視圖	功能說明
洞察	提供 AI 智慧建議、付款方式分析、訂單狀態總覽、久未收款明細與累積未收排行，協助老闆決定處理順序。

7.2 LINE Bot 使用流程

LINE Bot 補足 Dashboard 不適合手機即時操作的情境。客戶可以直接在 LINE 中瀏覽商品與下單；老闆則可在外出或配送途中查詢未付款清單、確認付款與標記送達。

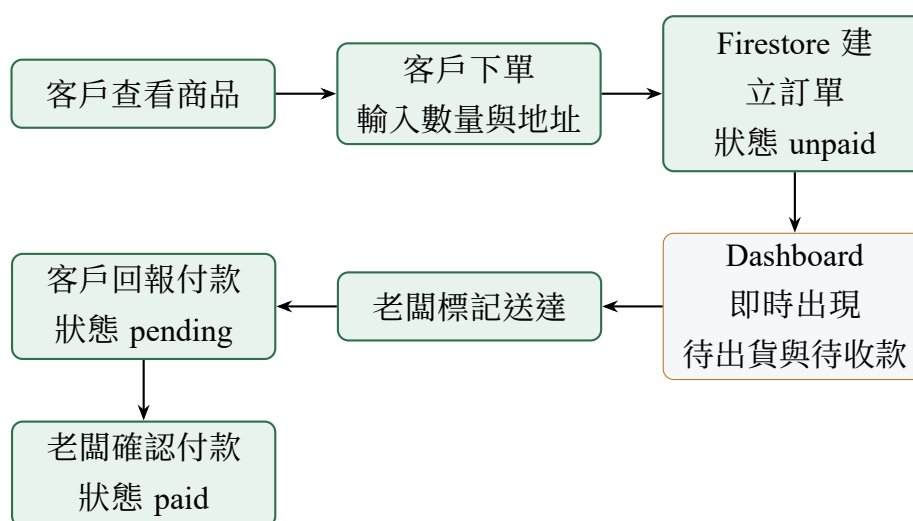


圖 4: LINE Bot 與 Dashboard 的訂單及付款流程

7.3 報表匯出

業主習慣使用 Excel 檔整理明細，因此系統在訂單作業、客戶與洞察視圖提供匯出列。使用者可勾選要輸出的欄位，產生 CSV、Excel 或列印 PDF。CSV 以 UTF-8 BOM 避免 Excel 開啟亂碼；Excel 使用 SheetJS 保留數值與日期型別；PDF 透過瀏覽器列印流程產生，方便保存或交給他人查看。

7.4 AI 智慧顧問

AI 智慧顧問位於洞察分頁。系統會整理 Firestore 中的訂單、商品與使用者資料，交由 Gemini API 分析並回傳結構化建議。若沒有設定 API key、API 呼叫失敗或網路異常，系統會自動切回本地規則引擎。這樣設計可確保 AI 功能是加值而非單點故障來源。

AI 建議依嚴重度分為 critical、warning、info、positive，常見建議包含：

- 金額大且拖欠時間久的未收款訂單。
- 待出貨累積過多，可能造成客戶等待。
- 某些客戶累積未收金額偏高，應優先對帳。
- 某類商品或客戶近期營收表現穩定，可作為經營參考。

8 系統正確性與測試驗證

8.1 型別與資料驗證

Dashboard 使用 TypeScript 定義 UserDoc、ProductDoc、OrderDoc 與 enum 型別，並透過 Firestore converter 將 Timestamp 轉為 Date。未知付款狀態會 fallback 為 unpaid，避免畫面因舊資料或異常資料白屏。

LINE Bot 端則以 services/schemas.py 中的 dataclass 驗證寫入資料，包含付款狀態、付款方式、金額合計、配送日期與未指派配送欄位的限制。兩端都使用 server timestamp 寫入關鍵時間欄位，降低資料時間不一致。

8.2 測試涵蓋

Dashboard repo 內包含 Vitest 測試，主要涵蓋 Firestore converter、付款狀態轉換、confirmOrderPayment、markOrderDelivered、客戶搜尋與 seed data。此類測試能確保最關鍵的商業邏輯在修改後不易回歸。

此外，專案提供以下開發指令：

```
npm run dev
npm run lint
npm run test
npm run build
```

其中 npm run build 會執行 TypeScript build 與 Vite 打包，可驗證前端型別與產物是否能正常生成。LINE Bot 端則可透過 python app.py 啟動 Flask webhook，並搭配 LINE Developer Console 或 tunnel 工具接收 callback。

8.3 已知限制

本專案目前仍有幾項限制，列為未來改善項目：

- Firebase Auth 與正式權限規則尚未完整導入，因此 Firestore rules 仍需在正式上線前配合 custom claim 與登入流程調整。

- 目前聚焦於訂單、出貨與帳款；BOM、材料耗損與成本結算尚未納入 MVP。
- AI 建議目前以訂單、商品、客戶資料為主，尚未加入使用者對建議的採納或回饋機制。
- LINE Bot 與 Dashboard 共用資料契約，未來若新增欄位，需要維持雙 repo 同步更新與測試。

9 與既有解決方案之差異與創新性

9.1 與 ERP 系統比較

ERP 的優點是功能完整，能整合會計、採購、庫存、生產、銷售與財務報表，適合流程標準化、部門分工明確的大型企業。然而堯順工程行目前最需要的不是完整流程控制，而是快速掌握訂單、送達與收款。本系統因此刻意不複製 ERP 的所有功能，而是聚焦於小型工程行每天真正會用到的資訊。

表 7: 既有 ERP 與本系統差異

既有 ERP 常見特性	本系統設計取向
導入與操作流程複雜，需完整建檔。	以訂單、商品、客戶與付款狀態為最小資料集合，降低使用門檻。
適合大型企業與標準化流程。	以小型工程行的 LINE 溝通、配送與追帳流程為核心。
功能多但許多模組用不到。	只設計業主目前最迫切的帳款、出貨、客戶歷史與報表功能。
對手機即時互動支援有限或需額外導入。	直接整合 LINE Bot，讓客戶與老闆可在熟悉工具中操作。
報表多為通用型分析。	加入久未收款、待確認付款、累積未收排行與工程行情境 AI 建議。

9.2 與 Excel、紙本或純 LINE 紀錄比較

Excel 與 LINE 都是業主熟悉的工具，但兩者各有缺點。Excel 雖然彈性高，卻需要自行設計欄位、公式與篩選；LINE 雖然即時，卻不適合長期查詢、彙總與分析。紙本紀錄更容易發生遺失、重複抄寫與資訊不一致。

本系統保留 LINE 的低門檻互動，同時將資料寫入 Firestore，讓 Dashboard 可以進行結構化查詢、統計與匯出。這使系統兼具「使用者熟悉」與「資料可管理」兩個優點。

9.3 創新性

本專案的創新不在於單一技術，而在於將多個技術組合成貼合業主情境的輕量資訊系統：

- **LINE 與 Dashboard 共用資料：**客戶在 LINE 下單或回報付款，Dashboard 立即呈現；老闆在 Dashboard 確認入帳，狀態也回寫同一份 Firestore。
- **付款確認狀態：**新增 `pending_confirmation`，解決「客戶說已付款」與「老闆確認入帳」之間的狀態落差。
- **小型工程行情境 AI：**AI prompt 以水泥與建材工程行為角色背景，分析重點放在未收款、配送與客戶風險，而不是通用商業報表。
- **自由文字配送標籤：**不強迫建立 driver 使用者角色，而是允許老闆輸入送貨人員或車號，符合小型團隊彈性分工。
- **可匯出且可追溯：**在不要求業主捨棄 Excel 的情況下，將 Excel 變成報表輸出，而不是主要資料庫。

10 對企業預期產生之效益

10.1 作業效率提升

舊流程需要翻閱紙本訂單、搜尋 LINE 紀錄、人工核對金額與狀態，再逐一編輯催帳訊息。導入系統後，老闆可直接打開 Dashboard 或 LINE Bot，查看待收款清單、待出貨清單與待確認付款。系統將原本分散的資訊整理成可排序、可篩選、可操作的工作清單，預期可降低每日重複查找與確認的時間。

10.2 帳款風險降低

多元付款方式與非固定收款人是業主最明確的痛點之一。本系統透過付款狀態、付款方式、入帳時間與待確認狀態，讓每筆款項都有清楚生命週期。久未收款與累積未收排行能協助老闆優先處理金額大或拖延時間長的客戶，降低漏收與現金流壓力。

10.3 資訊透明度提升

訂單品項、配送地址、送貨標籤、送達時間、付款方式與客戶歷史都集中於 Firestore。當老闆想查詢某位客戶的歷史訂單，或確認某筆訂單是否送達，不再需要依賴記憶或翻找訊息。資料透明度提升後，也更容易交接給其他員工或家人協助管理。

10.4 決策品質提升

Dashboard 的 KPI、付款方式分析、狀態總覽與 AI 建議能讓業主掌握營運趨勢。雖然目前尚未進入完整成本分析，但系統已先建立訂單與收款資料基礎，未來若要擴充 BOM、材料耗損或單位成本，也已有可延伸的資料來源。

11 專案啟發與未來展望

11.1 專案啟發

本專案讓我們體會到，資訊系統設計不一定是追求功能越多越好，而是要找出使用者真正願意每天使用的流程。業主曾嘗試 ERP，但因流程過重而難以持續使用，顯示系統成敗關鍵不只在技術，也在於操作成本是否符合企業規模。

在開發過程中，我們不斷在完整性與實用性之間取捨。例如「driver」角色在理論上可以做得很完整，但實務上業主更需要的是能快速輸入送貨人員或車號的自由欄位；付款流程也不是只有已付與未付，而是存在客戶已回報、老闆未確認的中間狀態。這些細節都來自訪談與實作中的需求校正。

11.2 未來展望

1. **LINE Login 與 LIFF 下單表單**：未來可加入 LIFF 表單，讓客戶用手機完成更完整的多品項下單、地址填寫與訂單確認。
2. **正式權限與 Firestore Rules**：導入 Firebase Auth 與 custom claims，讓 boss 與 customer 的資料讀寫權限更完整。
3. **成本與材料耗損模組**：延伸本次未處理的 BOM、材料耗損與成本透明問題，讓系統能估算單位成本與毛利。
4. **自動催帳與推播排程**：在老闆確認規則後，讓 LINE Bot 定期整理未付款清單或發送提醒，但仍保留人工確認避免過度打擾客戶。
5. **AI 回饋機制**：加入「有用、無用、已處理」等回饋，讓 AI 建議逐步貼近業主決策習慣。

6. **資料視覺化擴充**：加入月營收趨勢、客戶貢獻度、產品銷售組合與付款週期分析，形成更完整的經營儀表板。

12 交付內容與執行方式

12.1 專案交付內容

依評分說明，期末專案繳交壓縮檔應包含最終簡報、書面報告、系統相關程式碼與執行說明、訪談紀錄表與證明。本專案建議交付內容如下：

- 正式簡報：資管導正式簡報 (1).pdf
- 書面報告：本 LaTeX 原始檔與編譯後 PDF。
- Dashboard 程式碼：React/Vite 專案與 README。
- LINE Bot 程式碼：<https://github.com/OogaryoO/yaoshun>
- 訪談紀錄與證明：訪談逐字稿、照片、訊息截圖或錄音檔等應一併放入壓縮檔。

12.2 Dashboard 執行方式

```
npm install
cp .env.example .env.local
npm run dev
```

需在 `.env.local` 中設定 Firebase Web app 的 `VITE_FIREBASE_*` 變數。若要啟用 Gemini AI，則需設定 `VITE_GEMINI_API_KEY`；未設定時系統會自動使用本地規則引擎。

12.3 LINE Bot 執行方式

```
pip install -r requirements.txt
python app.py
```

LINE Bot 需設定以下環境變數：

- `LINE_CHANNEL_ACCESS_TOKEN`
- `LINE_CHANNEL_SECRET`
- `GOOGLE_CREDENTIALS_JSON`
- `BOSS_LINE_ID`

正式部署時需將 Flask 的 /callback endpoint 設為 LINE Developers Console 的 Web-hook URL。

13 結論

堯順工程行的問題並非缺乏大型系統，而是缺乏一個足夠簡單、能被日常使用、又能把 LINE 訊息與帳款資料轉為可管理資訊的工具。本專案以訪談中最明確的痛點為核心，建立了智慧儀表板與 LINE Bot 共用 Firestore 的整合系統，讓訂單、配送、付款與客戶歷史能被即時查詢與分析。

相較於既有 ERP，本系統更貼近小型工程行的工作節奏；相較於 Excel、紙本或純 LINE，本系統提供結構化資料、即時同步、報表匯出與 AI 分析。雖然目前仍有權限正式化、成本模組與 LIFF 介面等未來工作，但本系統已完成核心 MVP，能有效回應業主在訂單出貨與帳款追蹤上的主要需求。